

Shikha Nukala
Email: desigirl@umich.edu

Overview and Justification

I chose the project <https://github.com/publiclab/simple-data-grapher/> from GitHub. It is a small JavaScript project that lets users turn CSV files into graphs that can be embedded on other websites. Public Lab is a civic and environmental group, and this tool helps people visualize data for public use, which fits the “social good” theme of the assignment. The repository is beginner friendly and has many “first-timers” and “good first issue” labels. It also includes a clear CONTRIBUTING.md file with setup instructions. Because it is small and has active issues, it is easier to set up and work on within one month. This makes it a good choice for learning and making a real contribution through GitHub pull requests.

Successful Build

These are the exact steps I followed to successfully build and run the project on a local system (macOS, Linux, or Windows WSL2).

1. Install prerequisites: Node.js (v14 or higher), npm (comes with Node), xGit
2. Clone the repository: git clone <https://github.com/publiclab/simple-data-grapher.git>
3. npm install
4. npm run build
5. npm start
6. Verification of correct build output

```
Compiled successfully.  
  
File sizes after gzip:  
  
70.22 KB  build/static/js/2.5c5d9f78.chunk.js  
2.23 KB   build/static/js/main.2b5badbd.chunk.js  
1.08 KB   build/static/css/main.7a68c368.chunk.css  
792 B     build/static/js/runtime-main.23644d0a.js
```

Tasks Descriptions

Task 1 (High priority, easy, about 2–6 hours)

- Title: Add space in the button for clarity — Issue #88
- Description: This is a small UI fix. The button text “Create Spreadsheet” is missing a space before the plus icon, making it look cramped. You just need to add a space in the HTML string so it reads “Create Spreadsheet <i class=“fa fa-plus”>.” The issue is labeled “first-timers-only” and points to the exact file and line. After editing, rebuild the project and check in the browser that the button looks right.

- Why it fits: It is a simple, low-risk task that helps you learn the project flow, from editing to making a pull request.

Task 2 (Secondary priority, moderate, ~8–24 hours)

- Title: *Resolve linting errors* — Issue #50.
- Description: The repo has linting/style issues (JS code style warnings or errors). The task is to run the linting tool configured in the project (e.g., ESLint), address the flagged problems in the source code, and ensure the project passes lint checks. This may include small refactors and adding / updating the linter config to align with project style. Add unit or smoke tests if appropriate to demonstrate no regression.
- Why it fits: Improves code quality and CI feedback for the project; small but larger than Task 1 and provides good evidence of QA practices in the HW reports. Lint fixes are well scoped and typically straightforward but can take time if many issues exist.

Task Links

- Task 1: <https://github.com/publiclab/simple-data-grapher/issues/88>
- Task 2: <https://github.com/publiclab/simple-data-grapher/issues/50>

Requirements:

Task 1 — Add space in button

- Functional: Edit the file where the button string is created (the issue points to dist/PublicLab.Grapher.js, line numbers are given). The plus icon should be separated from the text: change Create Spreadsheet<i ...> → Create Spreadsheet <i ...>.
- Quality: Build must succeed; dev should confirm in a browser that the button now renders correctly without breaking layout. Provide the exact changed lines in the PR and a screenshot of the UI.

Task 2 — Resolve linting errors

- Functional: Run the repo's linter (e.g., ESLint) and fix the flagged issues in source files. Optionally update linter config if maintainers prefer (explain and propose changes in PR).
- Quality: Ensure npm run lint (or equivalent) returns zero errors (or only acceptable warnings as pre-agreed with maintainers). Add a short note in PR describing the rule changes/fixes. Run quick manual UI smoke tests to ensure no behavioral regressions.

Initial time plan

- Week 1 (Days 1–7) — about 7 hours total
 - Day 1 (1 hr): Clone the repo, read README and CONTRIBUTING.md, run the app locally, confirm it builds.
 - Day 2 (1 hr): Reproduce the UI bug from issue #88 and make a new branch.

Day 3 (1 hr): Fix the bug, rebuild, test, and make a pull request.

Buffer time (1–2 hrs): Write a good PR description and message the maintainers.

- Week 2 (Days 8–14) — about 8–10 hours

Days 8–9 (2–3 hrs): Run the linter, note the errors, and plan which ones to fix.

Days 10–12 (4–6 hrs): Fix the issues, test, and commit changes.

Day 13 (1 hr): Open a pull request with a clear explanation.

- Week 3 (Days 15–21) — about 4–6 hours

Use this week to respond to maintainer feedback, make any requested changes, add screenshots, and finalize pull requests.

Total estimated effort: around 20–24 hours working alone. If working with a partner, total time doubles to about 48 hours so each person still contributes about 24 hours.

Short risk assessment

One possible issue is that another contributor might start working on the same task. To avoid this, comment on the GitHub issue right away to claim it. Another risk is build problems caused by Node version differences. You can fix this by using the same Node version listed in `package.json` or using `nvm`. If the linting task becomes too large, start by fixing a few important problems and suggest doing the rest in future pull requests. Finally, if the project has rules against AI help, that could be a problem, but the repository's `CONTRIBUTING.md` does not forbid using AI tools. Still, I will mention in the pull request if AI was used to write or edit anything.

Initial process plan

I plan to use a simple, iterative approach, similar to Agile. I will make small, focused changes in separate branches and open one pull request per issue. The steps are: fork the repo, create a branch (for example the `fix/issue-88-space-btn`), make the changes, and open a pull request. I will follow the `CONTRIBUTING.md` instructions and respond to maintainer feedback. For testing, I will check the UI manually in the browser, run `npm run lint`, and include before-and-after screenshots. I will use GitHub issues for communication and ask for help or a review when needed.

Task scope justification

Task 1: The single-line spacing fix is self-contained and touches only one generated HTML string. It meets the assignment's requirement that the task modify source code (the dist file is generated from source) and will exercise the full contribution flow (fork → branch → PR → review). It's small enough to complete quickly but gives evidence of understanding the build process and PR etiquette. GitHub

Task 2: Linting error resolution requires reading and modestly refactoring source files, running local checks, and optionally updating linter config — a small but nontrivial code quality task. It is

typical of maintenance work and is appropriate to demonstrate QA, tests, and code comprehension — and sized to be achievable within the one-month timeframe.

I inspected the repository's CONTRIBUTING.md and did not find any statement forbidding the use of AI tools.